ApexPlanet Cybersecurity Internship

Task 3: Web Application Security & Testing

Full Report

Item	Details
Name	Rupashi Sethi
Role	Cybersecurity & Ethical Hacking Intern, ApexPlanet Software Pvt Ltd
Task	Task 3: Web Application Security & Testing
LinkedIn	https://www.linkedin.com/in/rupashi-sethi-4505a936a/
GitHub repository	rupa-code/Apexplanet-task-3
Video walkthrough	https://www.linkedin.com/feed/update/urn:li:activity:7525074463856418816/

Introduction

This report documents my practical work for Task 3 of the ApexPlanet Cybersecurity & Ethical Hacking Internship. I used the intentionally vulnerable Damn Vulnerable Web Application (DVWA) hosted on my Metasploitable2 virtual machine to practise common web application security tests. The work covered SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), local file inclusion (LFI), request inspection with Burp Suite, and Apache security-header configuration.

All testing was performed in my own isolated VirtualBox lab. Screenshots in this report are taken from the supplied practical evidence. Where a screenshot does not show a complete exploit or a verified result, the report states that limitation rather than assuming success.

Task Deliverables

Deliverable	Where to find it
DVWA vulnerability testing	Sections 3-6 of this report (Figures 2-7).
Burp Suite request analysis	Section 7 (Figures 8-10).
Apache security-header configuration	Section 8 (Figures 11-14).
GitHub repository and demo video	Links will be added after upload and repository push.

1. Lab and Scope

The practical work was carried out against DVWA inside my existing private lab. The target was Metasploitable2, which was previously configured on a VirtualBox Host-Only network. The same lab setup is used in my earlier internship task reports.

Item	Details
Attacker / testing workstation	MahaKali (Kali Linux), running the browser and Burp Suite.
Target application	DVWA (Damn Vulnerable Web Application) hosted on Metasploitable2.
Target VM address	192.168.56.102 (same target used in the earlier lab reports).
Virtual network	VirtualBox Host-Only network; intentionally vulnerable target kept within the private lab.
Web server shown in evidence	Apache/2.2.8 (Ubuntu) with PHP 5.2.4-2ubuntu5.10.

2. Testing Approach

- Open DVWA in the browser and confirm the selected security level before each test.
- Use controlled test inputs against the lab application and observe the page output or browser alert.
- Use Burp Suite Proxy and HTTP history to inspect requests and responses; configure Intruder only for controlled lab testing.
- Configure Apache response headers, run the configuration test, and inspect a later HTTP response.

Scope note: the screenshots include some incomplete or intermediate steps. The findings below distinguish between a demonstrated result, a configuration attempt, and an item that needs additional evidence.



Figure 1: Metasploitable2 target VM running in VirtualBox during the practical work.

3. SQL Injection Testing

I used the DVWA SQL Injection module to observe how an unsafe input can alter a database query. At Low security, a tautology-style input was accepted and the page returned multiple user records instead of a single expected record.

The Low-security evidence includes a lookup for user ID 1, which returned the admin record, and a second request that returned five records: admin, Gordon, Hack, Pablo and Smith. This is consistent with a successful SQL injection demonstration in the deliberately vulnerable lab.

Test	Observed result	Security note
Low security	The injected input returned multiple user records (five entries are visible in the evidence).	Input is not safely constrained or parameterised.
High security	A separate screenshot shows the High security setting and no user records in the output.	This particular screenshot suggests the displayed request did not return rows; it does not prove every injection path is blocked.

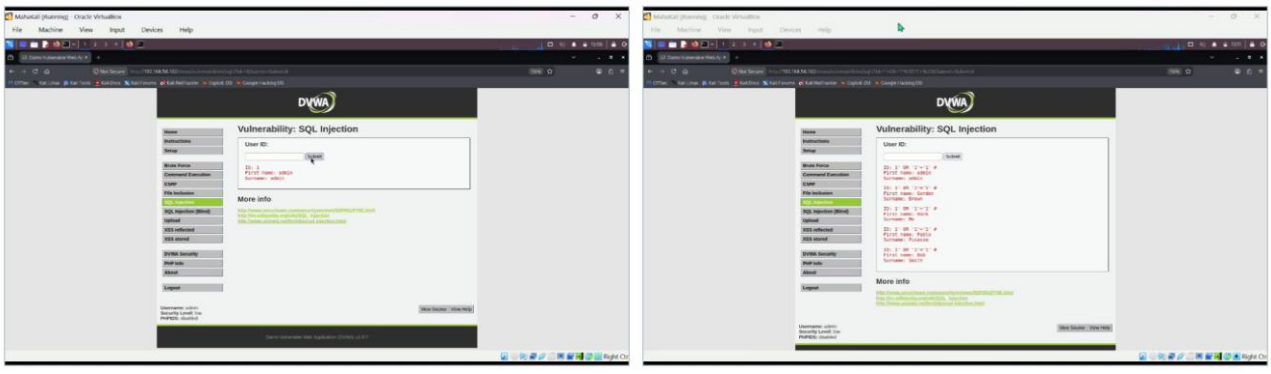


Figure 2: DVWA SQL injection at Low security: single-record lookup and a tautology-style input returning five user records.

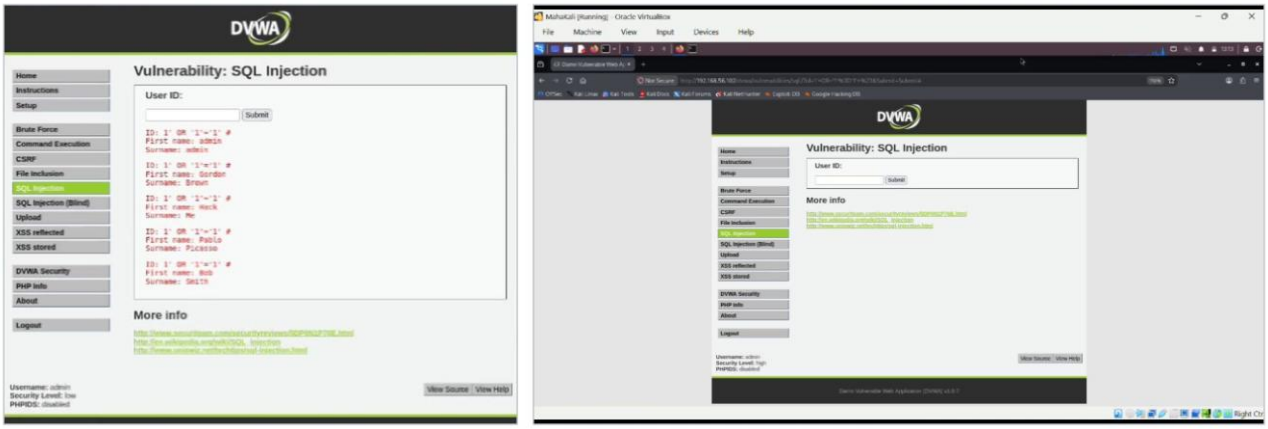


Figure 3: Low-security result alongside the separate High-security SQL injection page.

Observation: the test illustrates the risk of building SQL statements directly from user-controlled input. A production application should use parameterised queries, server-side input validation, least-privilege database accounts and consistent error handling.

4. Cross-Site Scripting (XSS) Testing

I tested both Reflected XSS and Stored XSS in DVWA. The browser alert dialogs in the screenshots show that script content supplied through the test fields was executed by the browser in the lab application context.

4.1 Reflected XSS

The Reflected XSS test used a script-based input. The result page displayed the supplied text and a browser alert labelled “XSS” appeared. This demonstrates that the response reflected untrusted input without adequate output encoding in the Low-security test.

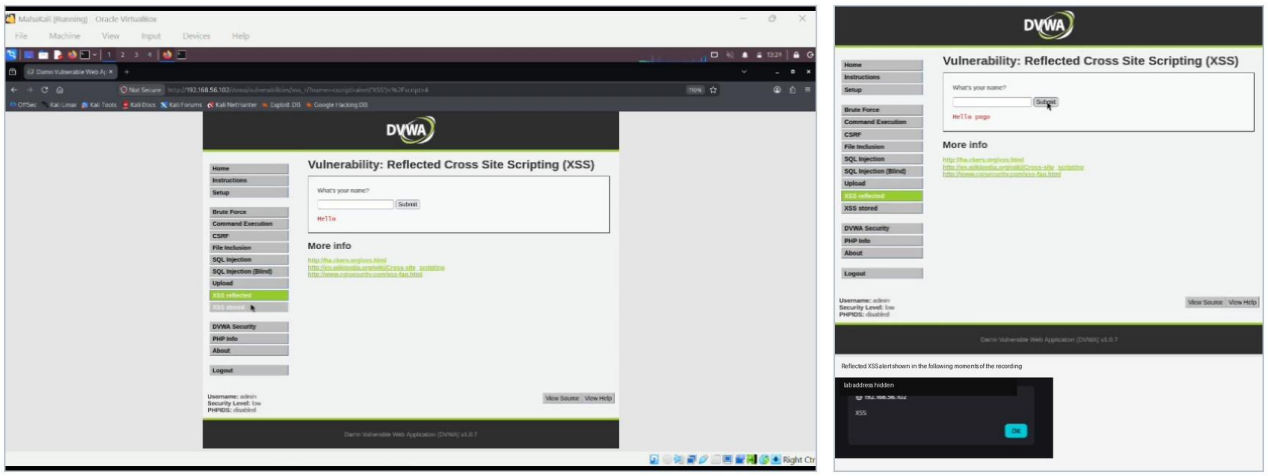


Figure 4: Reflected XSS test input and the browser alert observed in DVWA.

4.2 Stored XSS

For Stored XSS, a script-based message was entered through the guestbook form. A later view of the guestbook displayed the stored content and triggered an alert labelled “Stored XSS”. This shows that the payload persisted and executed when the stored content was rendered.

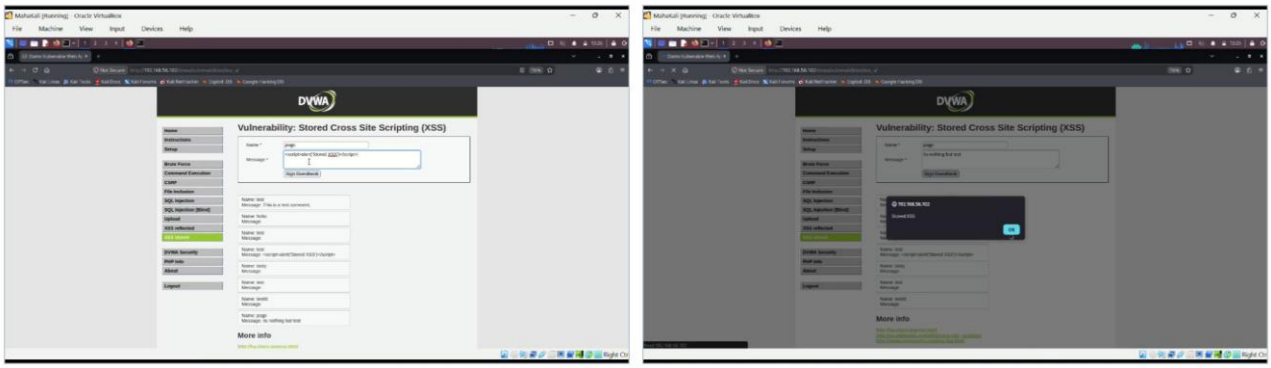


Figure 5: Stored XSS message entry followed by the alert when the stored message was displayed.

Recommended controls: encode output for the correct HTML/JavaScript context, validate input on the server, avoid unsafe DOM insertion, apply a restrictive Content Security Policy, and use HttpOnly/Secure/SameSite cookie attributes where appropriate. Input filtering alone is not a substitute for context-aware output encoding.

5. Cross-Site Request Forgery (CSRF)

I opened the DVWA CSRF password-change module with the security level set to High and inspected the form and its source. The available screenshots show the password-change form and related source code.

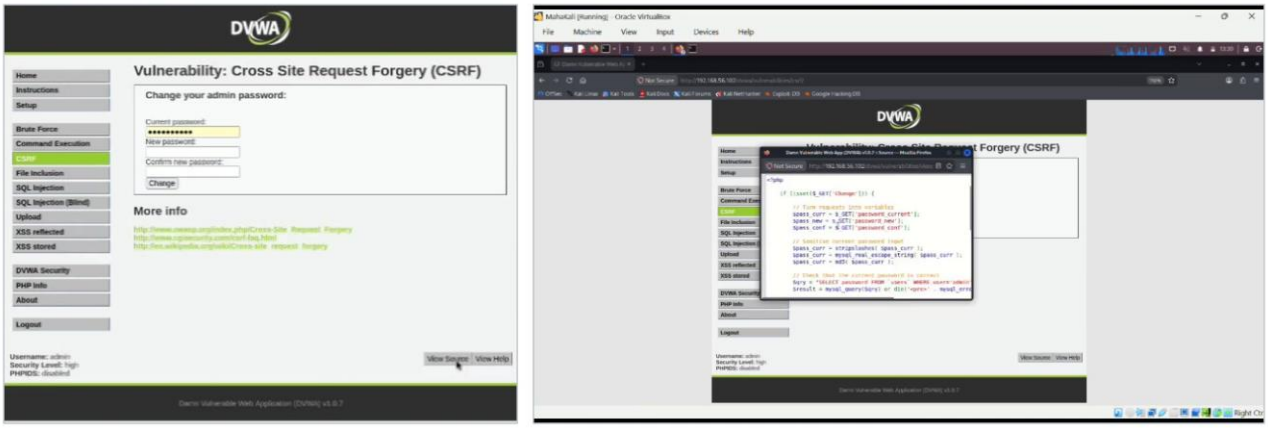


Figure 6: DVWA CSRF form at High security and the corresponding page/source view.

5.1 Evidence limitation

The screenshots do not show a completed forged cross-site request, a successful password change through CSRF, or a request being rejected because of an invalid or missing anti-CSRF token. Therefore, this report records the form/source inspection only and does not claim that a CSRF attack was successfully demonstrated or that token validation was verified.

5.2 Recommended protection

- Use unpredictable anti-CSRF tokens and validate them on every state-changing request.
- Use SameSite cookie settings as an additional layer, not as the only defence.
- Require re-authentication or confirmation for sensitive account changes, and avoid state changes through GET requests.

Additional evidence needed for a complete practical result: capture a controlled forged request in the lab and show the application rejecting a request with an absent or invalid token, alongside the valid-token case.

6. Local File Inclusion (LFI)

I opened the DVWA File Inclusion module at Low security and reviewed the available page output. A separate screenshot contains lines resembling local Linux account entries, along with PHP “Cannot modify header information - headers already sent” warnings.

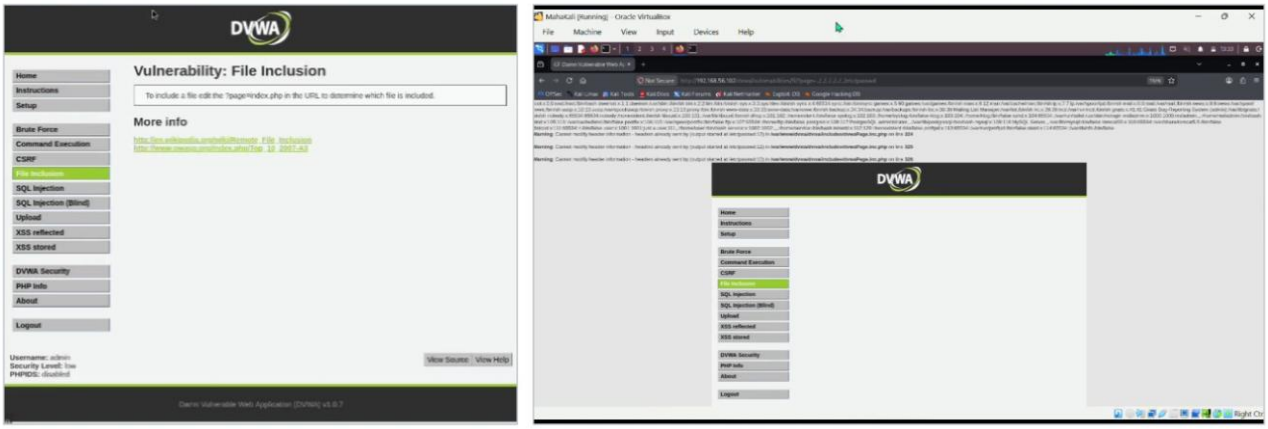


Figure 7: DVWA File Inclusion page and separate output containing /etc/passwd-style account entries with PHP header warnings.

6.1 Finding and limitation

The account-entry output is consistent with local file disclosure, but the screenshot is not a clean, controlled before-and-after demonstration. The page-opening screenshot alone does not prove a successful file read, and the available evidence does not include a High-security blocked test or a complete comparison of safe and unsafe file paths. The result is therefore treated as a preliminary LFI observation.

6.2 Recommended protection

- Do not pass user-controlled paths directly to file inclusion functions.
- Use an allowlist of permitted file identifiers and map them to fixed server-side paths.
- Canonicalise paths and verify that the resolved file remains inside the intended directory.
- Disable unnecessary remote file inclusion and run the web service with least privilege.

7. Burp Suite Request Inspection

I used Burp Suite to inspect HTTP traffic generated by the DVWA browser session. The screenshots show a login-related request intercepted in Proxy, a request/response entry in HTTP history and an Intruder payload configuration prepared for controlled testing.

7.1 Proxy and HTTP history

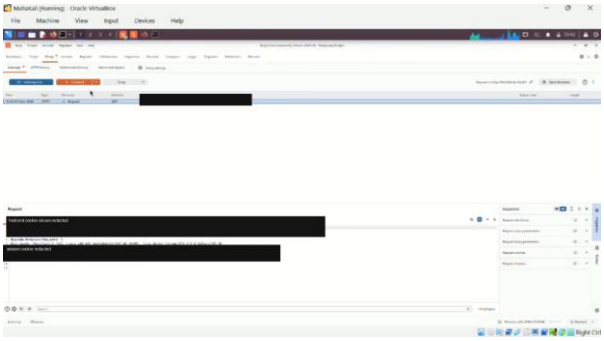


Figure 8: Burp Proxy intercept view.

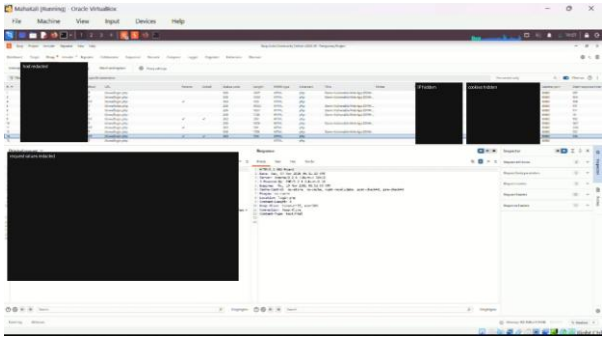


Figure 9: HTTP history with a 302 Found response.

A 302 response indicates that the server redirected the client. By itself, it is not evidence of a vulnerability; the request context and subsequent response must be assessed.

7.2 Intruder configuration

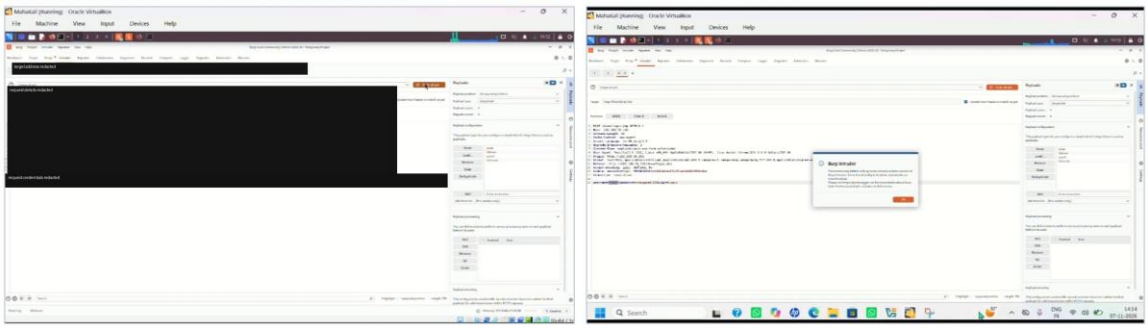


Figure 10: Burp Suite Intruder payload configuration and the later attack-started view.

The evidence shows payload positions/list configuration and an attack-started view, but it does not show a completed results table or a verified successful password guess. No account compromise or brute-force success is claimed. Any further Intruder testing should remain rate-limited and restricted to the authorised lab.

8. Apache Security Header Configuration

I checked the DVWA HTTP response before adding security headers, enabled Apache's headers module, and edited a local Apache configuration file named security-headers. The goal was to add response headers that reduce common browser-side risks.

8.1 Baseline response

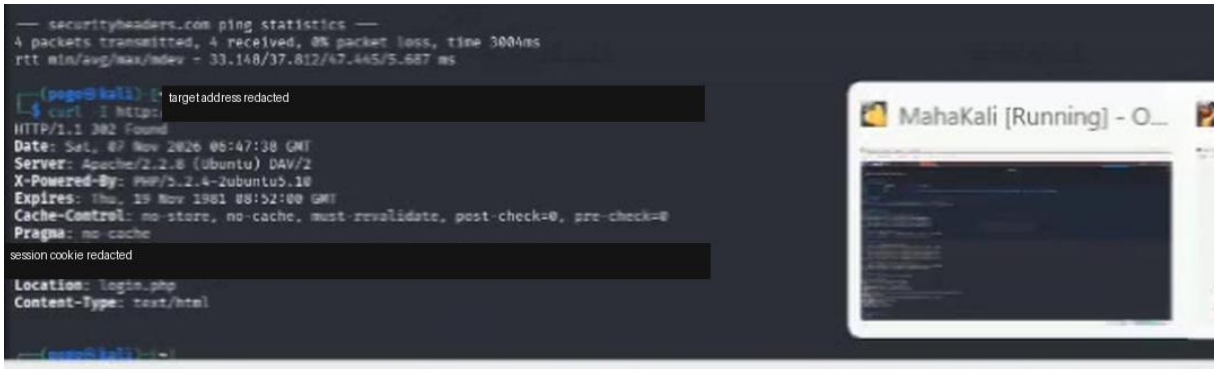


Figure 11: Initial HTTP response before the additional security headers were configured.

The baseline response showed Apache/2.2.8 (Ubuntu) DAV/2 and PHP 5.2.4-2ubuntu5.10. The captured response did not show the security headers that were added later.

8.2 Module and configuration

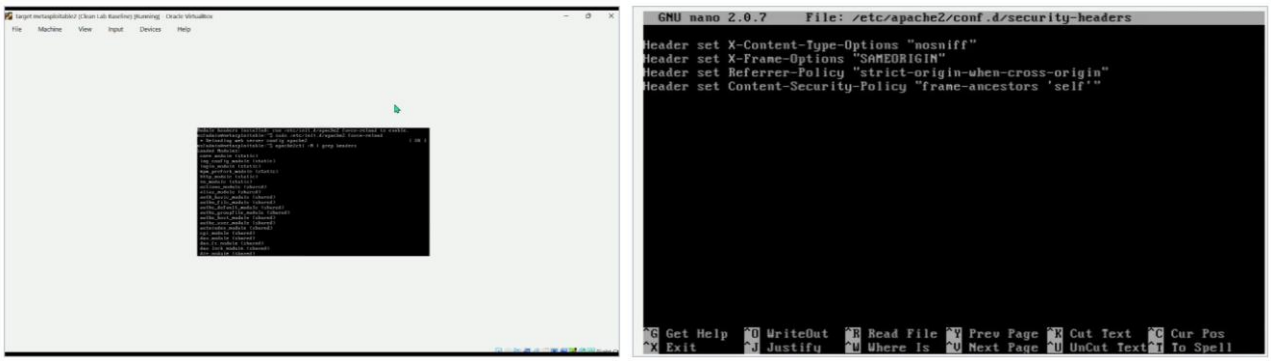


Figure 12: Apache headers module enablement and the security-header configuration file.

The configuration file shown in the evidence contains the following directives:

```
Header set X-Content-Type-Options "nosniff"
Header set X-Frame-Options "SAMEORIGIN"
Header set Referrer-Policy "strict-origin-when-cross-origin"
Header set Content-Security-Policy "frame-ancestors 'self'"
```

These settings request MIME-sniffing protection, same-origin framing, a restrictive referrer policy and a Content Security Policy frame-ancestors directive.8.3 Configuration verification

The recording shows an initial Apache configuration syntax error caused by a missing separator in a Header directive. The configuration was then corrected. A later configtest output reports “Syntax OK”. The same evidence also shows an attempted restart returning “httpd (no pid file) not running”, so the restart command itself was not confirmed as successful.

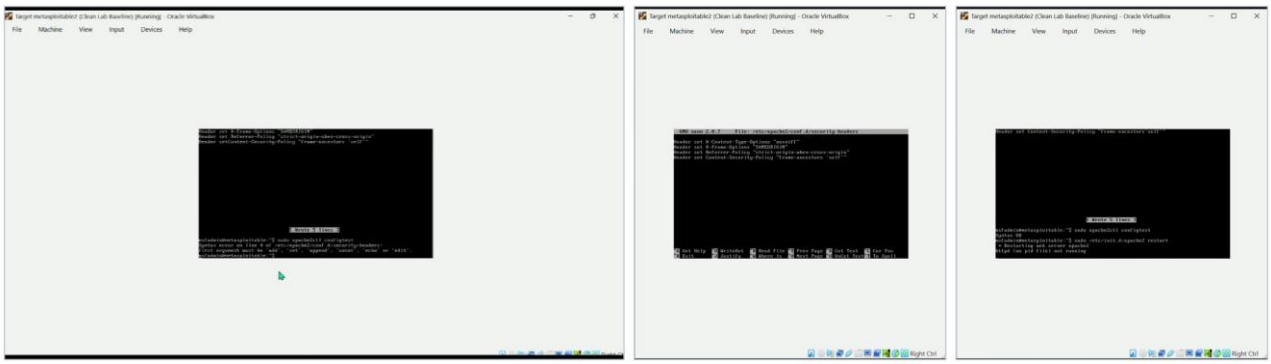


Figure 13: Initial configuration error, corrected configuration and Apache configtest/restart output.

8.4 Later HTTP response

A separate later capture shows an HTTP/1.1 200 OK response containing the configured headers: X-Content-Type-Options, X-Frame-Options, Referrer-Policy and Content-Security-Policy. It also displays X-XSS-Protection: 1; mode=block. This later response confirms the headers were present in that response, but it should not be presented as proof that the earlier restart command succeeded.

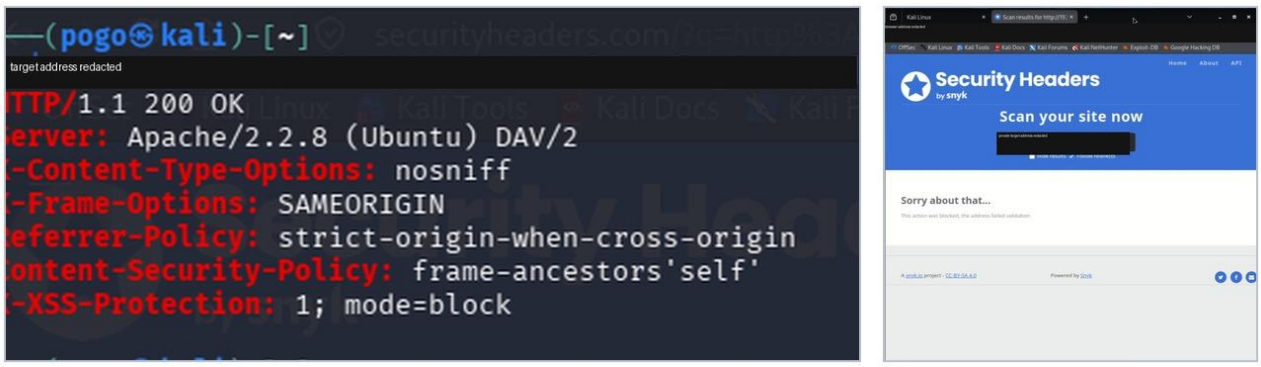


Figure 14: Later HTTP 200 response with security headers, alongside the external scanner refusing the private lab address.

The SecurityHeaders.com check did not complete because the target was a private lab address and the service rejected it. This is recorded as a testing limitation, not as a successful external security rating.

9. Summary of Findings and Recommendations

Area	Evidence-based observation	Priority / next action
SQL Injection	Low-security input returned multiple user records. A separate High-security screenshot showed no records for that test.	High: use parameterised queries; verify High-security handling with repeatable request evidence.
Reflected XSS	Script-based input triggered a browser alert in the response page.	High: apply context-aware output encoding and a restrictive CSP.
Stored XSS	Stored guestbook content triggered an alert when later displayed.	High: encode stored output and validate content on the server.
CSRF	High-security form and source were inspected; no completed forged request or token rejection is shown.	Evidence incomplete: capture valid-token and invalid-token request outcomes.
LFI	Separate output resembles local account-file content, but PHP warnings appear and no controlled high-security comparison is shown.	Medium/High: use allowlisted file IDs and canonical path validation; repeat with clean evidence.
Burp Suite	Proxy interception, HTTP history and Intruder configuration are shown; no verified Intruder result is shown.	Continue only in the lab; save completed results before claiming credential testing.
Apache headers	A later HTTP 200 response displays the configured security headers. Earlier restart attempt reported that httpd was not running.	Verify service reload/restart state, retest response headers and refine CSP beyond frame-ancestors.

Conclusion

In Task 3 I practised web application security testing against DVWA in my isolated Metasploitable2 lab. The available evidence demonstrates SQL injection at Low security, Reflected XSS and Stored XSS, as well as request inspection with Burp Suite and the configuration of Apache security headers. The LFI evidence is suggestive but not fully conclusive, while the CSRF and Intruder activities need additional result screenshots before they can be described as completed demonstrations.

This task helped me understand how insecure input handling can expose database records or execute scripts in a browser, how an intercepting proxy helps examine web requests, and how response headers provide additional browser-side protections. All tests were kept within my own private lab.